

is extremely fragmented. The only practical thing, it appears, is to target the lowest common denominator. This is not so. Two concepts that are worth discussing in this context are “polyfills” and “shims”!

A “polyfill” is basically just a fancy name for a library of some kind that provides the functionality of a particular HTML5 specification for a browser that does not natively provide that support. For example, older versions of Internet Explorer - up until version 8 - do not support the use of the HTML5 “canvas” tag. If being able to use that functionality in a page is critical to the app then it might make sense to use a polyfill library that provides the same functionality through some other means – perhaps via Silverlight or Flash plugins. The user experience will certainly not be the same as what it would be had the feature been natively supported but the users aren’t completely out of luck either.

“Shims” are similar to polyfills in that they are libraries as well, except for the fact that they do not conform to any HTML5 specification. A good polyfill library would provide complete fidelity with the HTML5 spec in question (say, the Canvas API in the example cited before) where as a “shim” usually provides a workaround or provides a library that is at a higher level of abstraction. The “Amplify.js” library for instance solves the problem of client side storage using the best available option on a given browser. On browsers supporting the HTML5 DOM Storage or Indexed DB specs the library would leverage that and use alternatives on other browsers. A “shim” usually provides its own API for the functionality it provides and does not conform to any HTML5 spec even though it might leverage them when available.

Feature detection vs. Browser detection

It is a commonly used technique in web development these days to enable/disable functionality depending on the specific brand and/or version of browser a given user is running. This is usually achieved by inspecting the “user agent” string that browsers provide to identify themselves. In JavaScript for instance, it is common to find code such as the following:

```
• if (navigator.userAgent.indexOf("MSIE") == -1) {  
•     window.addEventListener("load", onLoad);  
• } else {  
•     window.attachEvent("onload", onLoad);  
• }
```

This is not a good idea because here you are making assumptions about